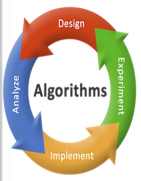
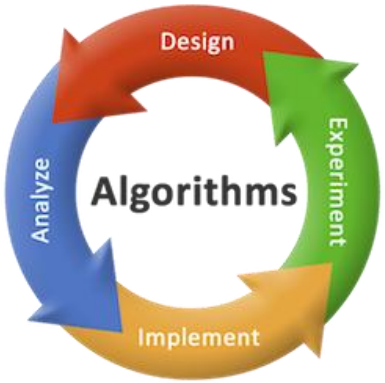
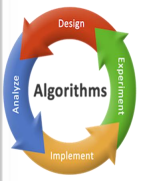
IQRA UNIVERSITY IU



Design and Analysis of Algorithms



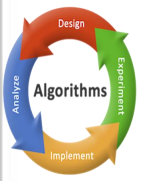
IQRA UNIVERSITY IU



Introduction to Algorithms

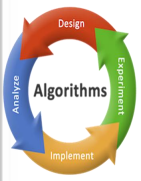
Greedy Algorithms

Greedy Algorithms



- A *greedy algorithm* always makes the choice that looks best at the moment
  - My everyday examples
    - Playing cards
    - Invest on stocks
    - Choose a university
  - The hope
    - A locally optimal choice will lead to a globally optimal solution

Introduction

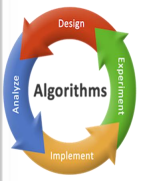


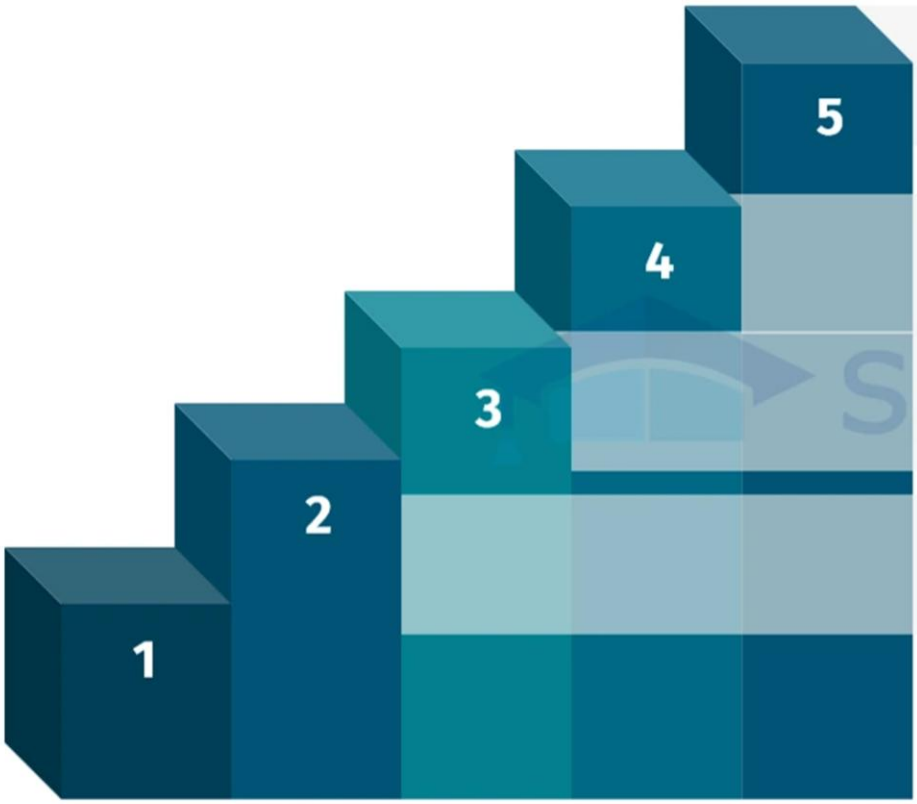
- Similar to Dynamic Programming
- It applies to Optimization Problem
- When we have a choice to make, make the one that looks *best right now*
  - Make a *locally optimal choice* in hope of getting a *globally optimal solution*
- Greedy algorithms don't always yield an optimal solution, but sometimes they do
  - For many problems, it provides an optimal solution much more quickly than a dynamic programming approach

IQRA UNIVERSITY IU

ScholarHat

Types of Greedy Algorithms

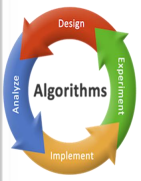




5. Huffman code algorithm
4. Prim’s algorithm
3. Kruskal’s algorithm
2. Dijkstra's algorithm
1. Ford Fulkerson algorithm

ScholarHat

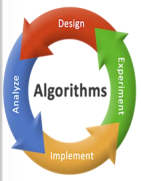
Greedy Algorithm Examples:



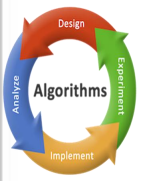
Examples of greedy algorithms are the best way to understand the algorithm. Some greedy algorithm real-life examples are:

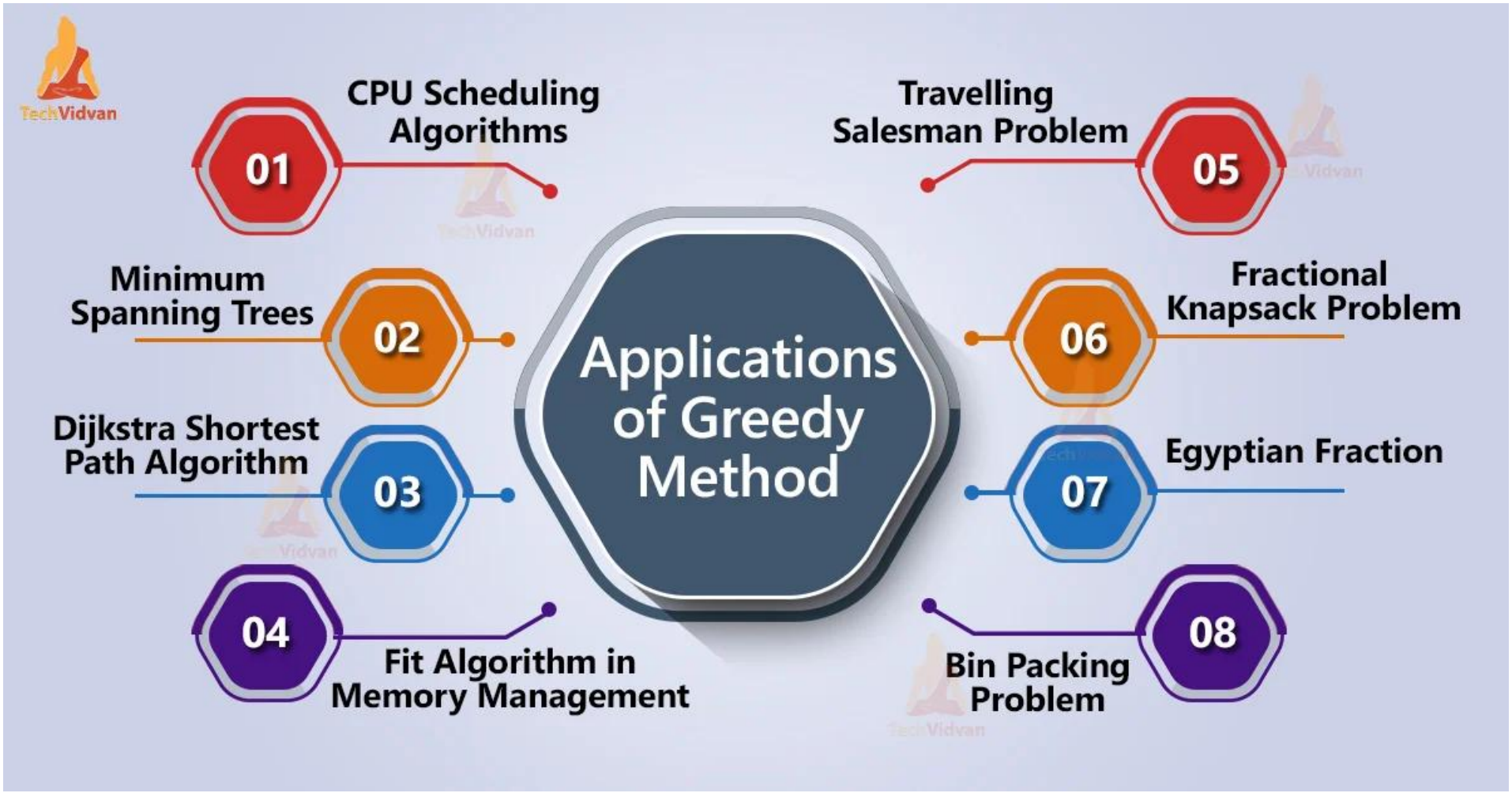
- **Fractional Knapsack:** Optimizes the value of items that can be fractionally included in a knapsack with limited capacity.
- **Dijkstra's algorithm:** Finds the shortest path from a source vertex to all other vertices in a weighted graph.
- **Kruskal's algorithm:** Finds the minimum spanning tree of a weighted graph.
- **Huffman coding:** Compresses data by assigning shorter codes to more frequent symbols.

What Is an Example of a Greedy Method in Real Time?



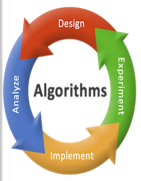
- In real-time applications, greedy methods are extensively utilized for their efficiency. For instance, in gaming and AI, they provide optimal solutions swiftly.
- In networking, they effectively manage data structures and enable real-time optimization.
- In machine learning, they expedite decision-making processes.
- Moreover, in scheduling and software engineering, they enhance performance by offering quick, albeit not always optimal, solutions.
- Such applications demonstrate the significant role of greedy techniques in various fields.


Design
Experiment
Implement
Analyze
Algorithms


TechVidvan
Applications of Greedy Method
01
CPU Scheduling Algorithms
02
Minimum Spanning Trees
03
Dijkstra Shortest Path Algorithm
04
Fit Algorithm in Memory Management
05
Travelling Salesman Problem
06
Fractional Knapsack Problem
07
Egyptian Fraction
08
Bin Packing Problem

IQRA UNIVERSITY IU

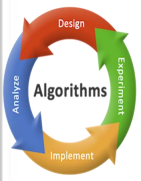
Pseudo code of Greedy Algorithm




```
Algorithm Greedy (a, n)
{
   Solution : = 0;
  for i = 0 to n do
  {
      x: = select(a);
     if feasible(solution, x)
    {
        Solution: = union(solution , x)
    }
       return solution;
  } }
```

IQRA UNIVERSITY IU

Greedy Algorithm to find Minimum number of Coins



Problem#1:

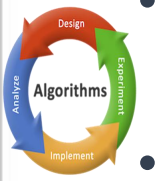
Given a value of **V** Rs and an infinite supply of each of the denominations {1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, The task is to find the minimum number of coins and/or notes needed to make the change?

Examples:

Input: V = 70
Output: 2
Explanation: We need a 50 Rs note and a 20 Rs note.

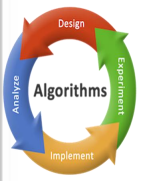
Input: V = 121
Output: 3
Explanation: We need a 100 Rs note, a 20 Rs note, and a 1 Rs coin.

Huffman's Algorithm



- It is a lossless data compressing technique generating variable length codes for different symbols.
- It is based on greedy approach which considers frequency/probability of alphabets for generating codes.
- It has complexity of (nlogn) where n is the number of unique characters.
- The length of the code for a character is inversely proportional to frequency of its occurrence.
- No code is prefix of another code due to which a sequence of code can be unambiguously decoded to characters.
- It is also known as **Huffman Encoding**.

Prefix Rule:

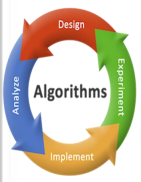


- Huffman Coding implements a rule known as a prefix rule.
- This is to prevent the ambiguities while decoding.
- It ensures that the code assigned to any character is not a prefix of the code assigned to any other character.

Major Steps in Huffman Coding:

There are two major steps in Huffman Coding-

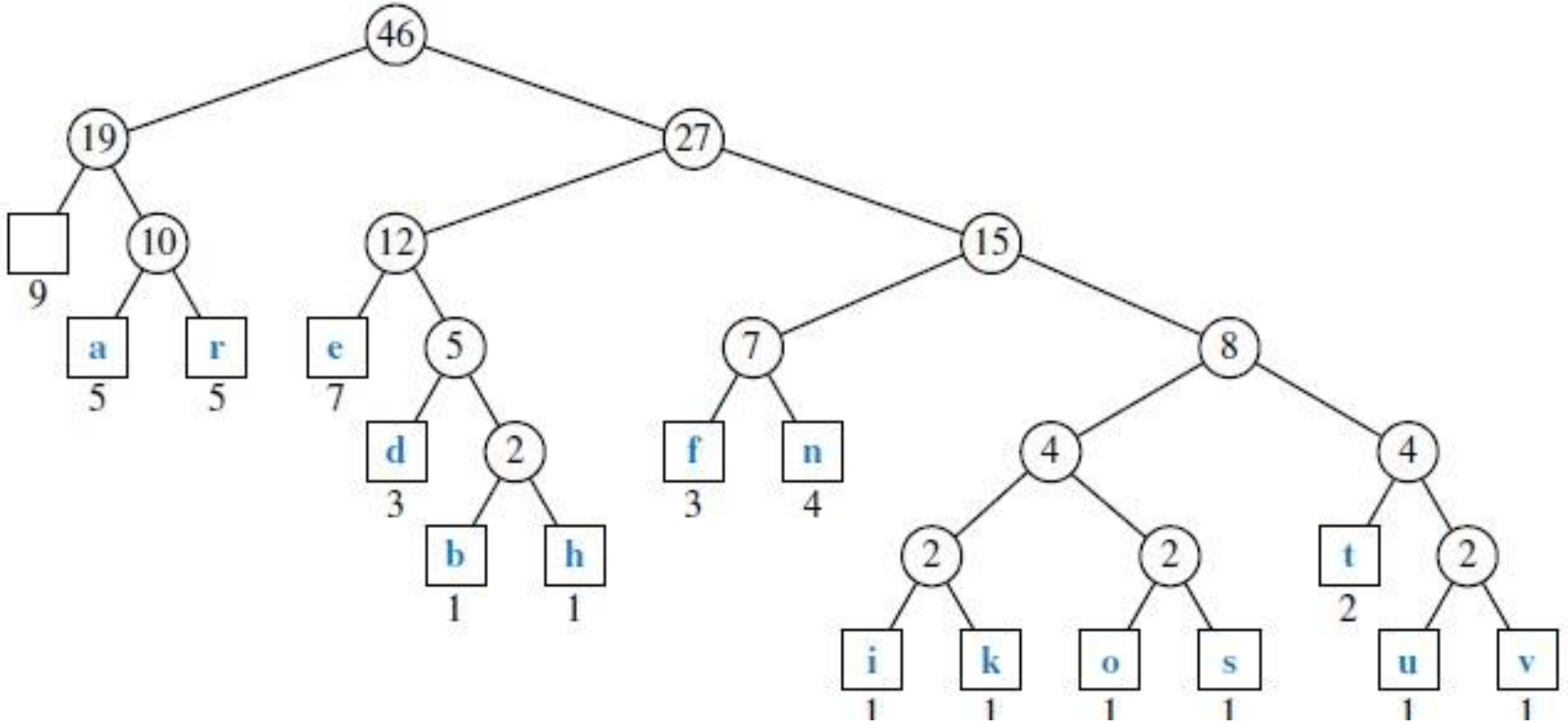
1. Building a Huffman Tree from the input characters.
2. Assigning code to the characters by traversing the Huffman Tree.



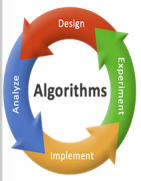
(a)

Character		a	b	d	e	f	h	i	k	n	o	r	s	t	u	v
Frequency	9	5	1	3	7	3	1	1	1	4	1	5	1	2	1	1

(b)

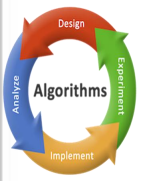


Algorithm:



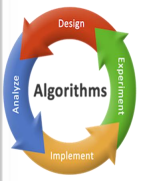
- The method which is used to construct optimal prefix code is called **Huffman coding**.
- This algorithm builds a tree in bottom up manner. We can denote this tree by **T**
- Let, |c| be number of leaves
- |c| -1 are number of operations required to merge the nodes. Q be the priority queue which can be used while constructing binary heap.

```
Algorithm Huffman (c)
{
        n= |c|
  Q = c
   for i<-1 to n-1
   do
        {
        temp <- get node ()
         left (temp] Get_min (Q) right [temp] Get Min (Q)
        a = left [templ b = right [temp] ]
        F [temp]<- f[a] + [b]
        insert (Q, temp)
        }
   return Get_min (0)
}
```



IQRA UNIVERSITY IU

Complexities:



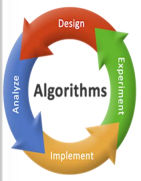
Time complexity:

- O(nlogn) where n is the number of unique characters. If there are n nodes, extractMin() is called 2*(n – 1) times. extractMin() takes O(logn) time as it calls minHeapify(). So, the overall complexity is O(nlogn).
- If the input array is sorted, there exists a linear time algorithm. We will soon be discussing this in our next post.

Space complexity: O(N)

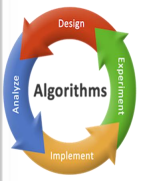
IQRA UNIVERSITY IU

Applications of Huffman Coding:



- Conventional compression formats like PKZIP, GZIP, etc. typically employ Huffman coding.
- Huffman Coding is used for data transfer by fax and text because it minimizes file size and increases transmission speed.
- Huffman encoding (particularly the prefix codes) is used by several multimedia storage formats, including JPEG, PNG, and MP3, to compress the files.
- Huffman Coding is mostly used for image compression.
- When a string of often recurring characters has to be sent, it can be more helpful.

Follow the steps below to implement the idea:

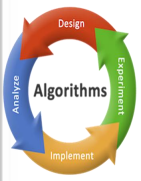


- Sort the array of coins in decreasing order.
- Initialize **ans** vector as empty.
- Find the largest denomination that is smaller than **remaining amount** and while it is smaller than the **remaining amount**:
  - Add found denomination to **ans**. Subtract value of found denomination from **amount**.

If amount becomes **0**, then print **ans**.

Important Formulas-

The following 2 formulas are important to solve the problems based on Huffman Coding-



Formula-01:

$$\text{Average code length per character} = \frac{\Sigma\,(\text{frequency}_i \times \text{code length}_i)}{\Sigma\,\text{frequency}_i}$$

$$= \Sigma\,(\text{probability}_i \times \text{code length}_i)$$

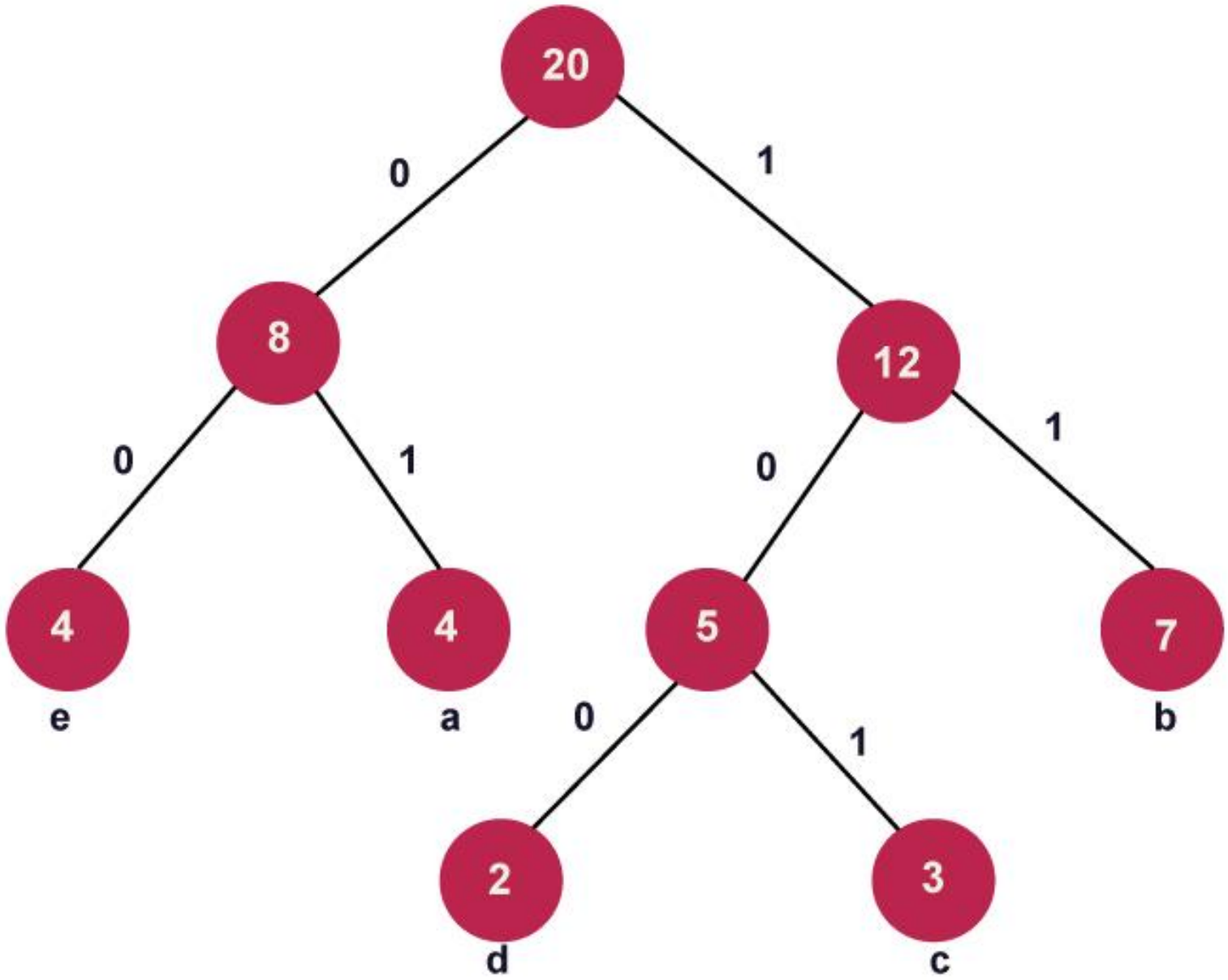
Formula-02:

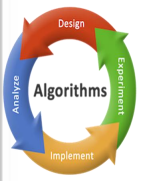
Total number of bits in Huffman encoded message

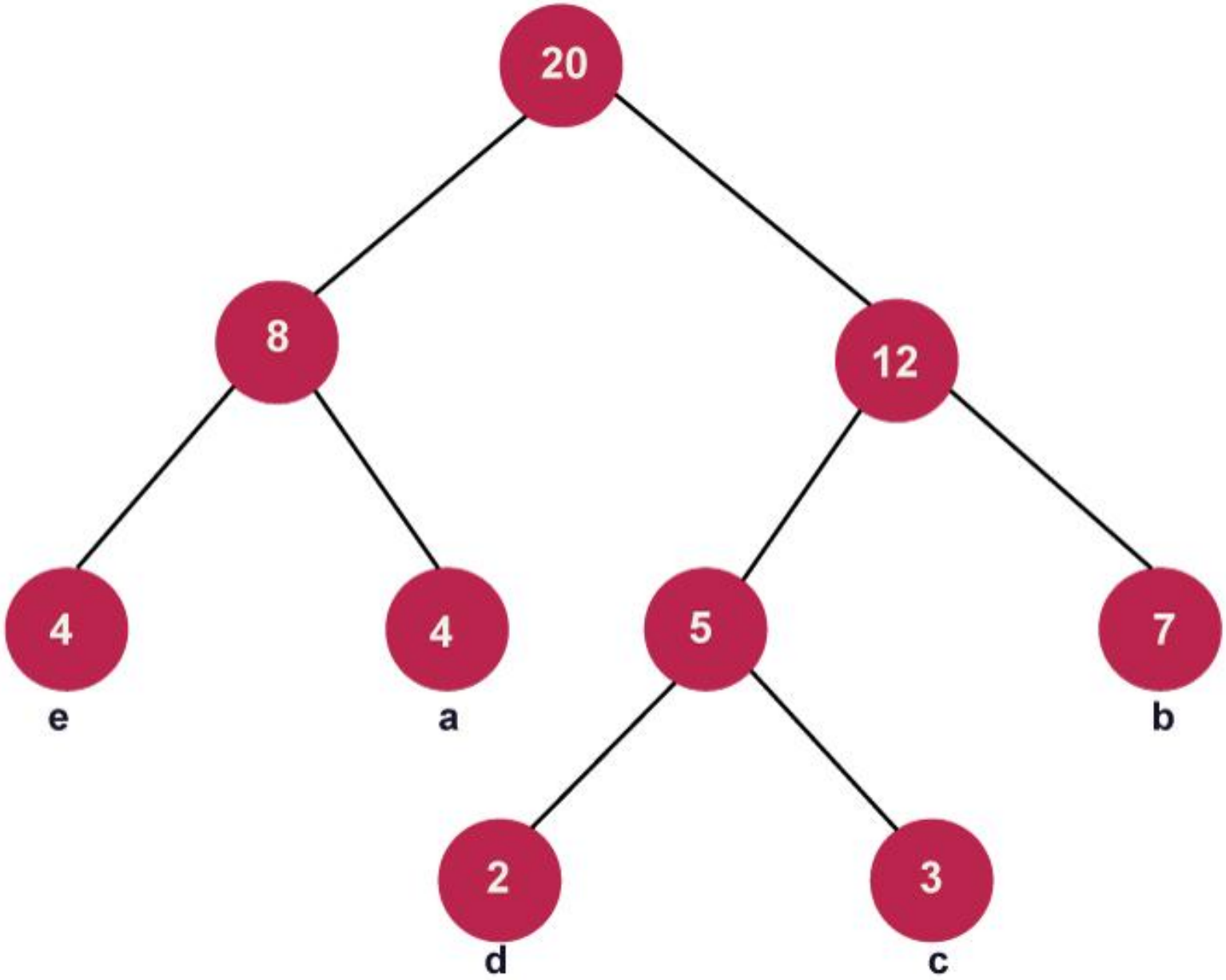
= Total number of characters in the message x Average code length per character = $\sum$ (frequency_i x Code length_i)

Examples of Huffman Coding

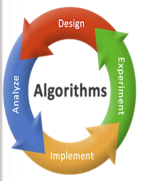
Let's use an illustration to explain the algorithm:




Design
Experiment
Analyze
Algorithms
Implement


20
8
12
4
4
5
7
e
a
b
2
3
d
c

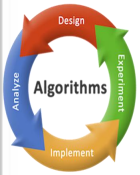
Algorithm for Huffman Coding



Step 1: Build a min-heap in which each node represents the root of a tree with a single node and holds 5 (the number of unique characters from the provided stream of data).



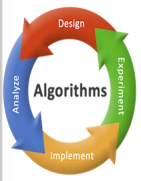
Step 2: Obtain two minimum frequency nodes from the min heap in step two. Add a third internal node, frequency 2 + 3 = 5, which is created by joining the two extracted nodes.

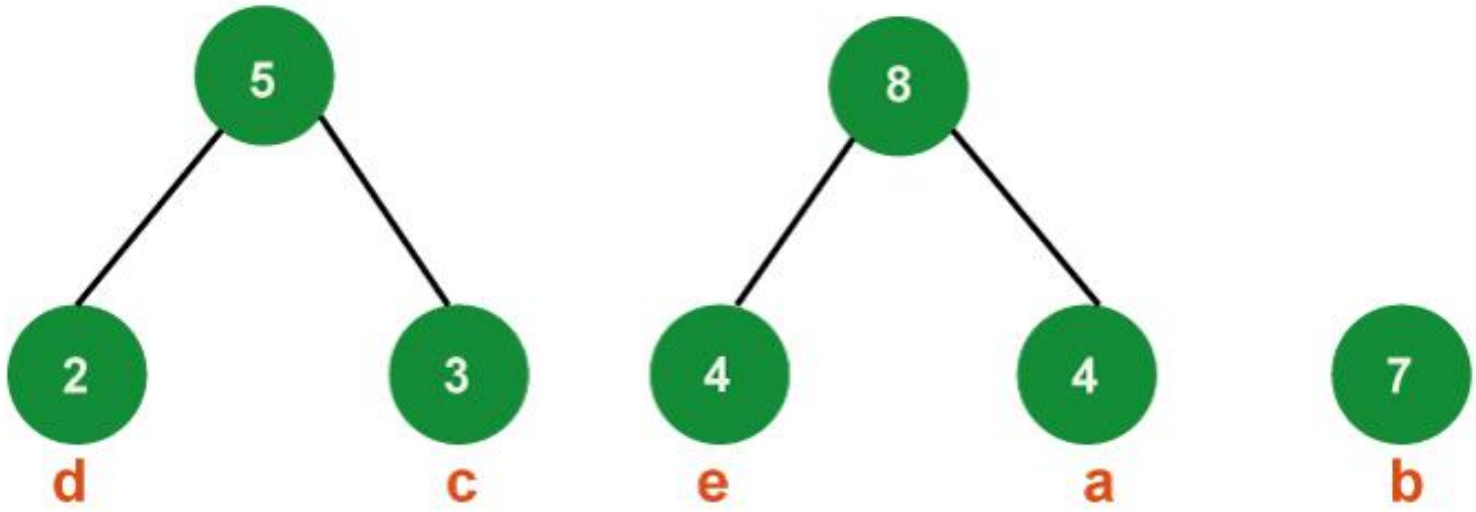




Now, there are 4 nodes in the min-heap, 3 of which are the roots of trees with a single element each, and 1 of which is the root of a tree with two elements.

Step 3: Get the two minimum frequency nodes from the heap in a similar manner in step three. Additionally, add a new internal node formed by joining the two extracted nodes; its frequency in the tree should be 4 + 4 = 8.

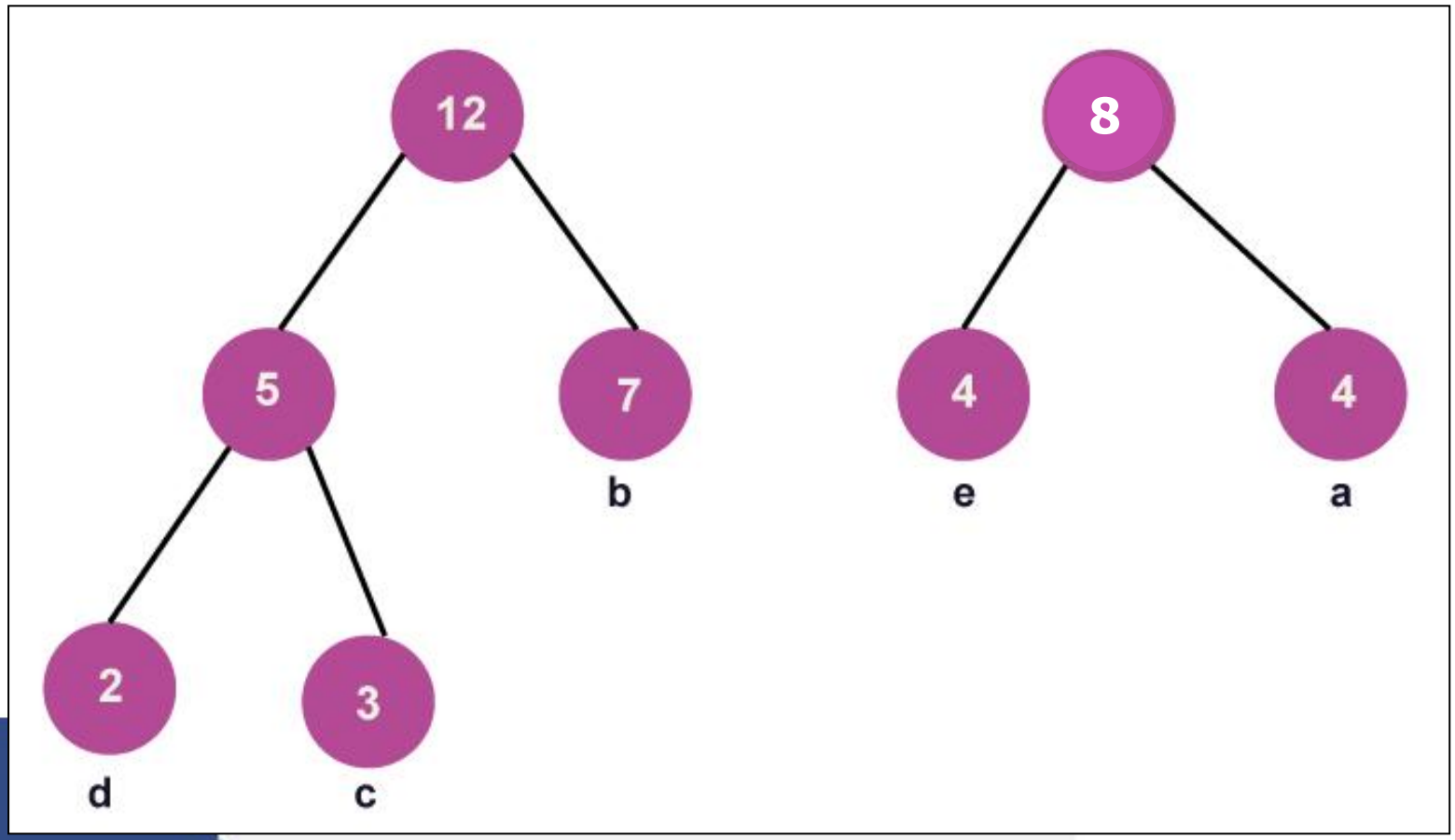


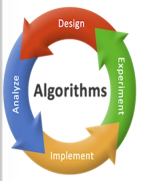


Now that the minimum heap has three nodes, one node serves as the root of trees with a single element and two heap nodes serve as the root of trees with multiple nodes.

Step 4: Get the two minimum frequency nodes in step four. Additionally, add a new internal node formed by joining the two extracted nodes; its frequency in the tree should be 5 + 7 = 12.

- When creating a Huffman tree, we must ensure that the minimum value is always on the left side and that the second value is always on the right side. Currently, the image below shows the tree that has formed:



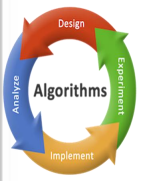


Step 5: Get the following two minimum frequency nodes in step 5. Additionally, add a new internal node formed by joining the two extracted nodes; its frequency in the tree should be 12 + 8 = 20.

Continue until all of the distinct characters have been added to the tree. The Huffman tree created for the specified cast of characters is shown in the above image.

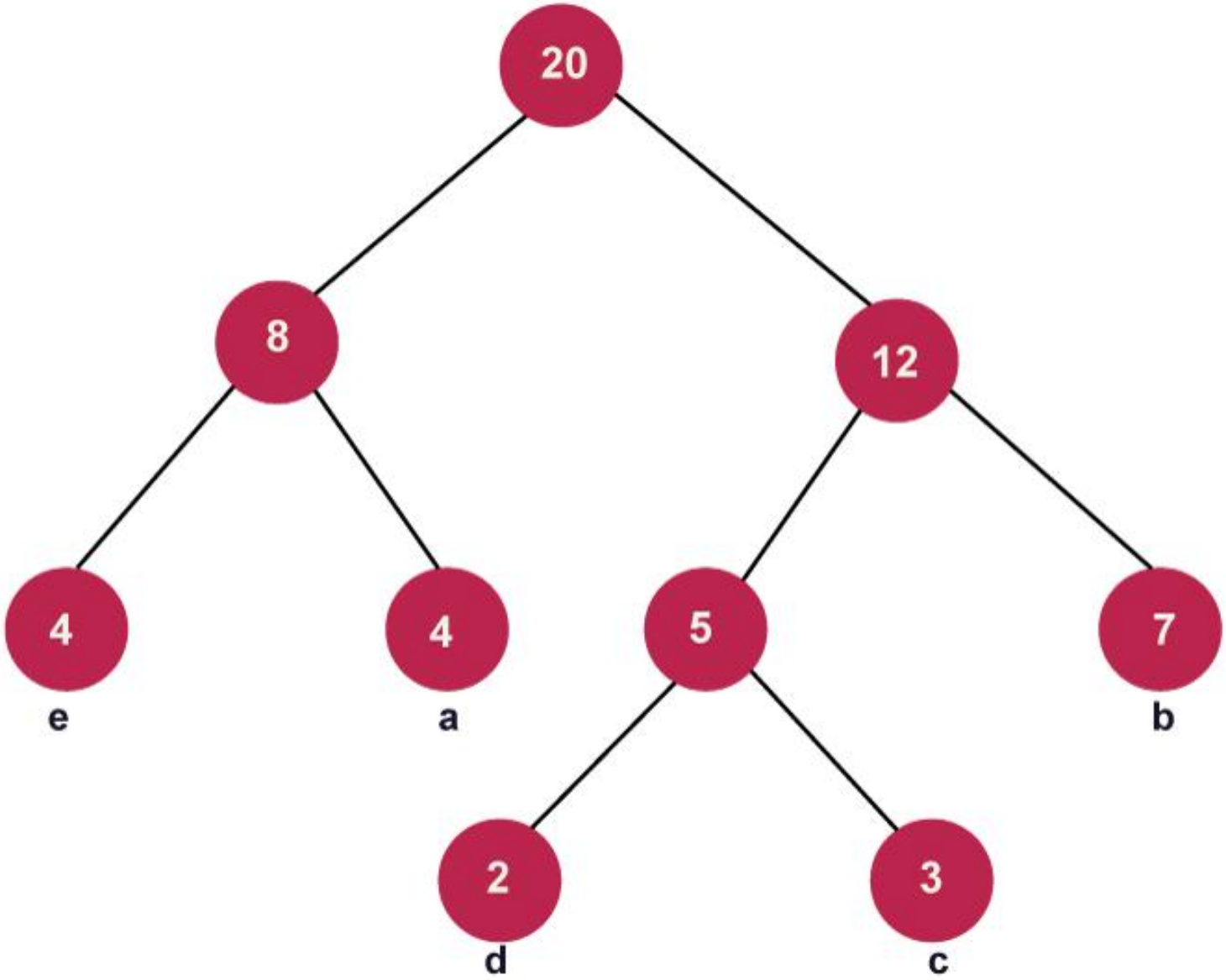
Now, for each non-leaf node, assign 0 to the left edge and 1 to the right edge to create the code for each letter.

Rules to follow for determining edge weights:



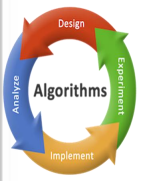
- We should give the right edges weight 1 if you give the left edges weight 0.
- If the left edges are given weight 1, the right edges must be given weight 0.
- Any of the two aforementioned conventions may be used.
- However, follow the same protocol when decoding the tree as well.

Following the weighting, the modified tree is displayed as follows:

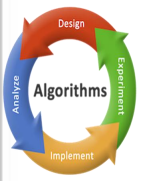


IQRA UNIVERSITY IU

Understanding the Code:

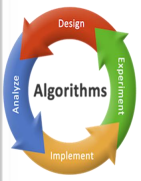


- We must go through the Huffman tree until we reach the leaf node, where the element is present, in order to decode the Huffman code for each character from the resulting Huffman tree.
- The weights across the nodes must be recorded during traversal and allocated to the items located at the specific leaf node.
- The following example will help to further illustrate what we mean:
- To obtain the code for each character in the picture above, we must walk the entire tree (until all leaf nodes are covered).
- As a result, the tree that has been created is used to decode the codes for each node. Below is a list of the codes for each character:



Character	Frequency/count	Code
a	4	01
b	7	11
c	3	101
d	2	100
e	4	00

Interval Scheduling: Greedy Algorithms Analysis of earliest-finish-time-first

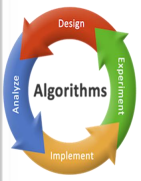


Greedy template.

Consider jobs in some natural order. Take each job provided it's compatible with the ones already taken.

- [Earliest start time] Consider jobs in ascending order of sj.
- [Earliest finish time] Consider jobs in ascending order of fj .
- [Shortest interval] Consider jobs in ascending order of fj - sj.
- [Fewest conflicts] For each job j, count the number of conflicting jobs cj . Schedule in ascending order of cj .

"A greedy algorithm is an algorithmic paradigm that follows the problem solving heuristic of making the locally optimal choice at each stage with the hope of finding a global optimum".

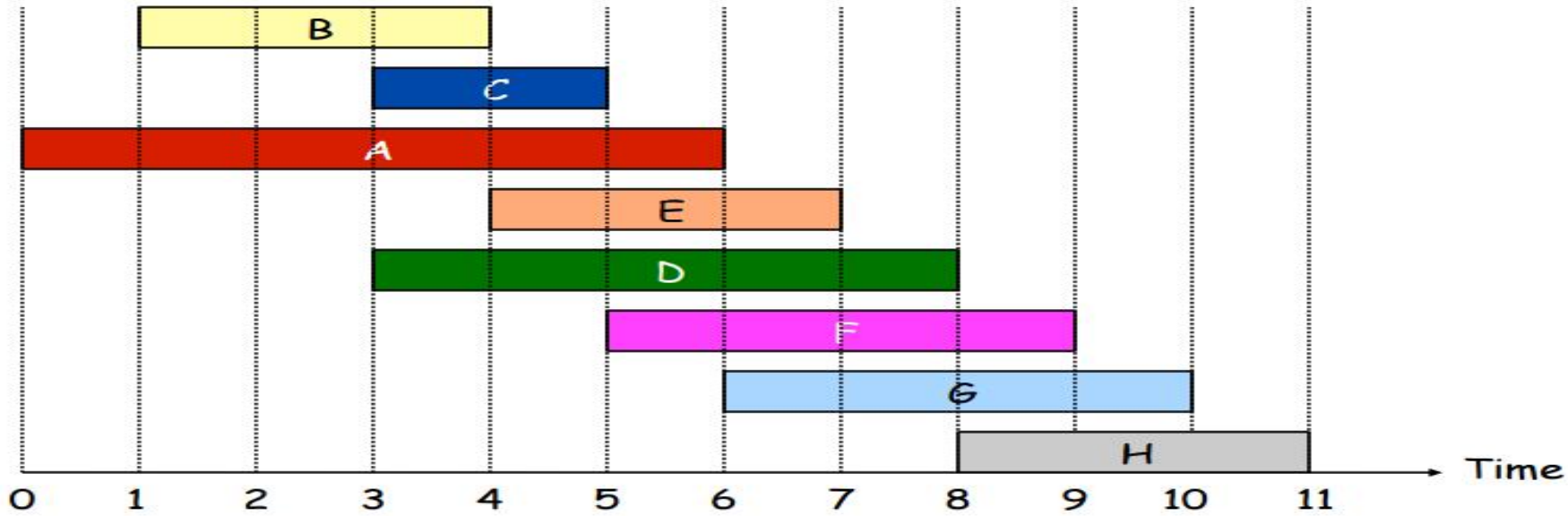


You should know that there are many cases where greedy algorithms are, in principle alone, not capable of finding the global optimum. Not for the problem we're here to talk about though!

As it turns out, there exists a greedy algorithm to solve the interval scheduling problem that can be proven to always find the optimal solution.

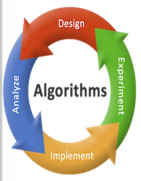
The optimal algorithm is 2 steps:

1. Sort the list of intervals (si,fi) by finishing time fi.
2. While elements remain in the list, add the soonest-ending interval compatible with the existing solution set to the solution



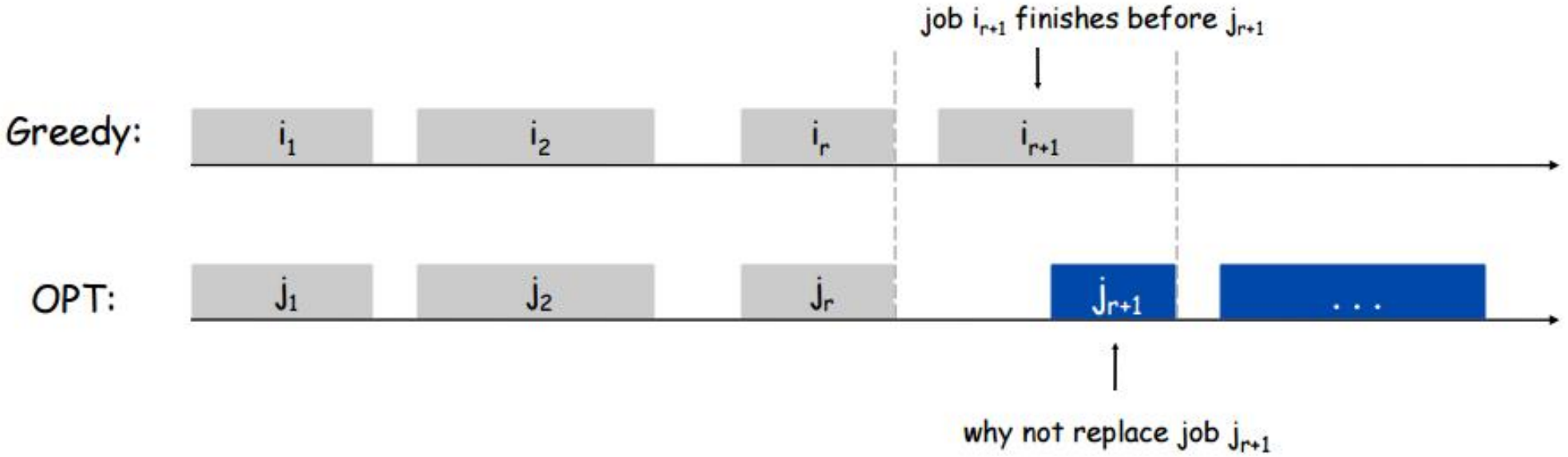


It's not obvious that this algorithm produces the optimal solution. It does, though. How do we prove this? By contradiction.

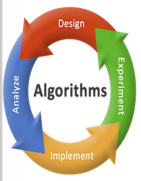


(by contradiction):

- Assume greedy is not optimal, and let's see what happens.
- Let i1, i2, ... i k denote set of jobs selected by greedy.
- Let j1, j2, ... jm denote set of jobs in the optimal solution with i 1 = j1, i2 = j2, ..., ir = jr for the largest possible value of r.



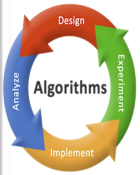
Interval Partitioning: Earliest-start-time-first Algorithm

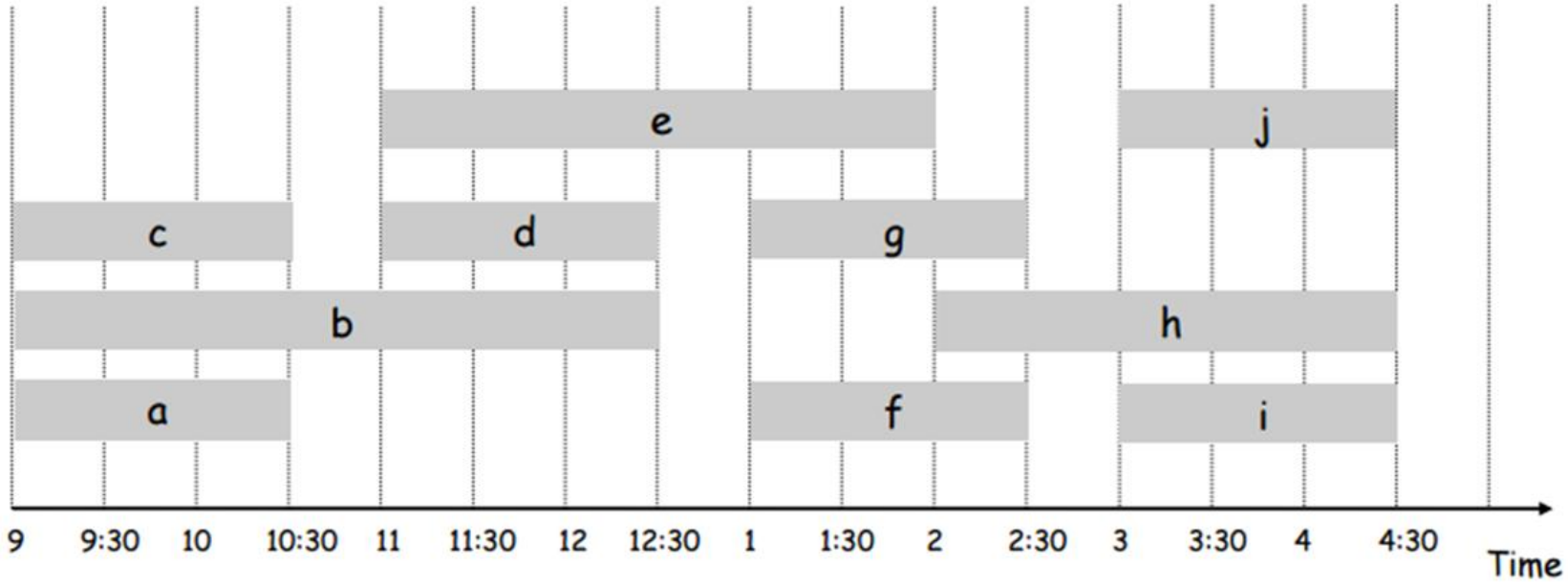


Interval partitioning:

- Lecture j starts at sj and finishes at fj.
- Goal: find minimum number of classrooms to schedule all lectures so that no two occur at the same time in the same room.

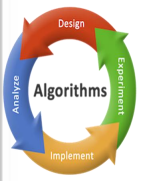
Ex: This schedule uses 4 classrooms to schedule 10 lectures.

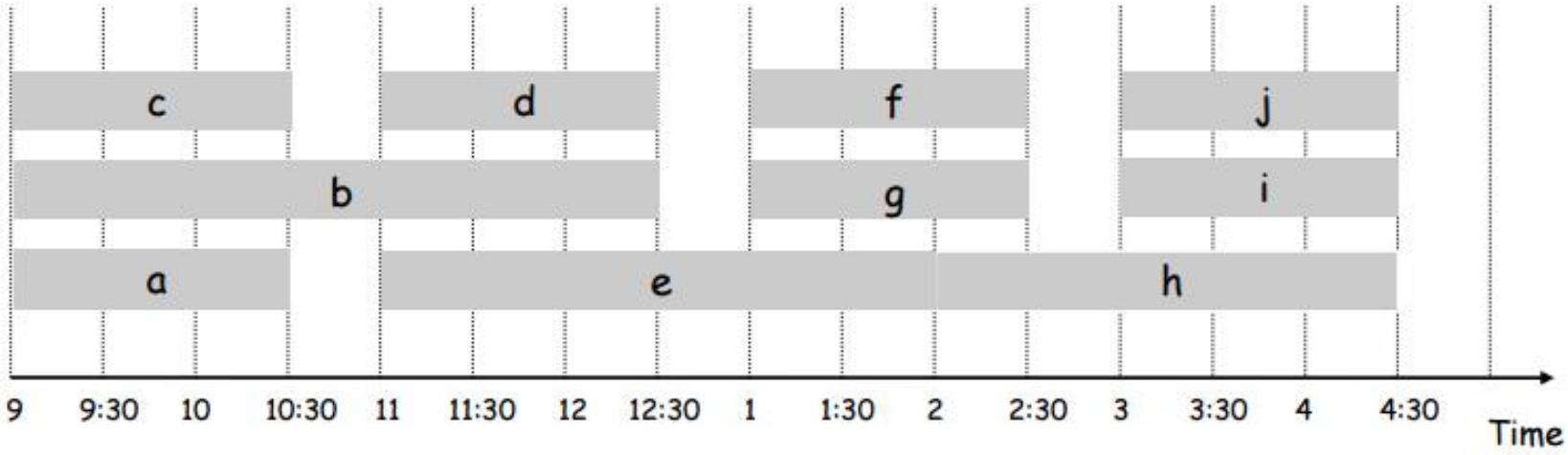


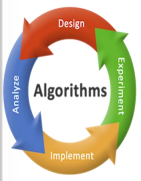


Notice that in the image above, there are at most 3 lectures that are running at every moment in time. This is called the **depth** of the set of intervals. Notice also that in the picture above, 4 classrooms are used, despite the depth of the set of intervals being 3. Below, a solution is shown that uses only 3 classrooms for the same set of interval

Ex: This schedule uses only 3



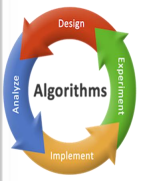




This is a better way to partition the intervals since it uses only 3 classrooms. Notice that the number of classrooms used is equal to the depth of the set of intervals. A good question is whether or not this is always possible? It's obvious that the number of classrooms needed is at least the depth of the set, but does there always exists a partitioning that uses exactly the number of classrooms as the depth of the set **and no more**?

Observation:

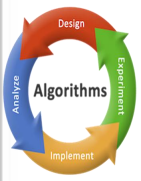
Greedy algorithm never schedules two incompatible lectures in the same classroom.



Theorem. Greedy algorithm is optimal.

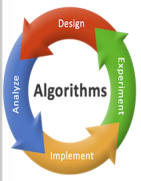
- Let d = number of classrooms that the greedy algorithm allocates. Classroom d is opened because we needed to schedule a job, say j, that is incompatible with all d-1 other classrooms.
- These d jobs each end after sj .
- Since we sorted by start time, all these incompatibilities are caused by lectures that start no later than sj .
- Thus, we have d lectures overlapping at time sj + ε.
- Key observation ⇒ all schedules use ≥ d classrooms. ▪

Scheduling to Minimize Lateness; No idle time



Imagine we have a bunch of jobs to do **one at a time** where each job has two properties: a duration and a deadline. We want to schedule these jobs so that the maximum amount time that any single job is past its deadline is minimized. We are **not** trying minimize total lateness, just *maximum* lateness. Here's a graphic to some up the situation:

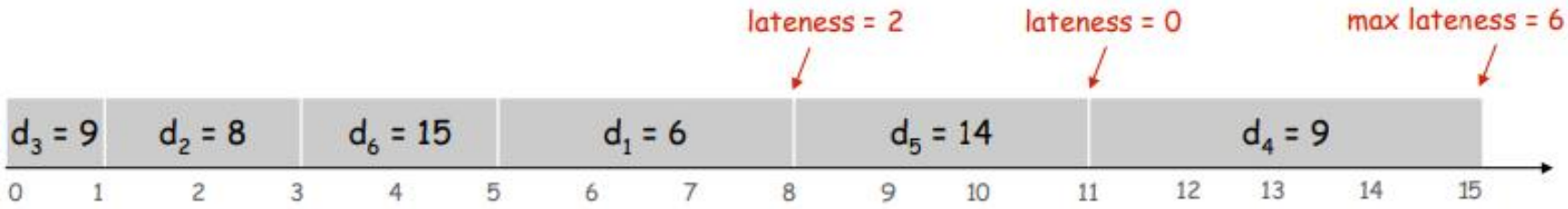
Minimizing lateness problem.



- Single resource processes one job at a time.
- Job j requires tj units of processing time and is due at time dj.
- If j starts at time sj, it finishes at time fj = sj + tj.
- Lateness: lj = max { 0, fj - dj }.
- Goal: schedule all jobs to minimize maximum lateness L = max lj.

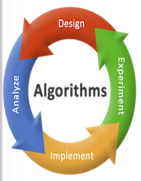
Ex:

	1	2	3	4	5	6
t_j	3	2	1	4	3	2
d_j	6	8	9	9	14	15

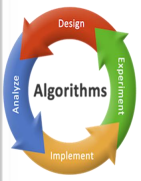


Greedy template. Consider jobs in some order.

- [Shortest processing time first] Consider jobs in ascending order of processing time tj .
- [Earliest deadline first] Consider jobs in ascending order of deadline dj .
- [Smallest slack] Consider jobs in ascending order of slack dj - tj .



Greedy template. Consider jobs in some order.



- [Shortest processing time first] Consider jobs in ascending order of processing time tj.

	1	2
t_j	1	10
d_j	100	10

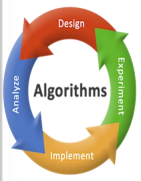
counterexample

- [Smallest slack] Consider jobs in ascending order of slack dj - tj.

	1	2
t_j	10	1
d_j	10	2

counterexample

Minimizing Lateness: No Idle Time

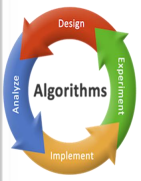


Observation. There exists an optimal schedule with no idle time.



Observation. The greedy schedule has no idle time

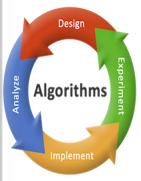
Greedy algorithm. Earliest deadline first




```
Sort n jobs by deadline so that d_1 ≤ d_2 ≤ … ≤ d_n


t ← 0
for j = 1 to n
    ( Assign job j to interval [t, t + t_j] )
    s_j ← t, f_j ← t + t_j
    t ← t + t_j
output intervals [s_j, f_j]
```

An Activity Selection Problem



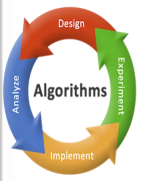
- The problem of scheduling several competing activities that require exclusive use of a common resource
  - *n* ***activities*** require *exclusive* use of a common resource
  - For example, scheduling the use of a classroom

- Set of activities $S = \{a_1, \ldots, a_n\}$
- a_i needs resource during period $[s_i, f_i)$
  - [ ) is a half-open interval
  - s_i = *start time* and f_i = *finish time*

- ***Goal***
  - Select the largest possible set of *non-overlapping*
  - (***mutually compatible***) activities

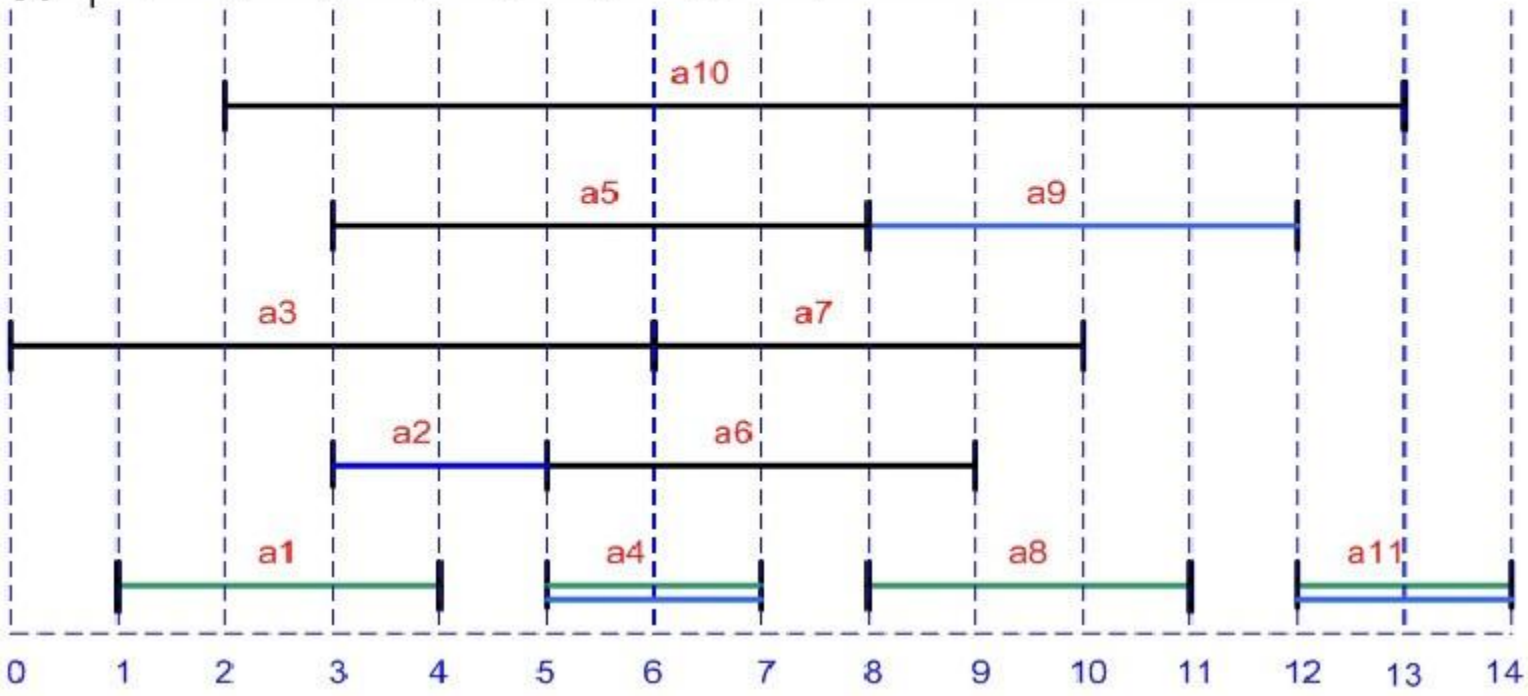
- ***Note:*** Could have many other objectives
  - Schedule room for longest time
  - Maximize income rental fees

An Activity Selection Problem



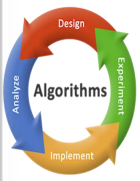
- Here are a set of start and finish times
  - What is the maximum number of activities that can be completed?

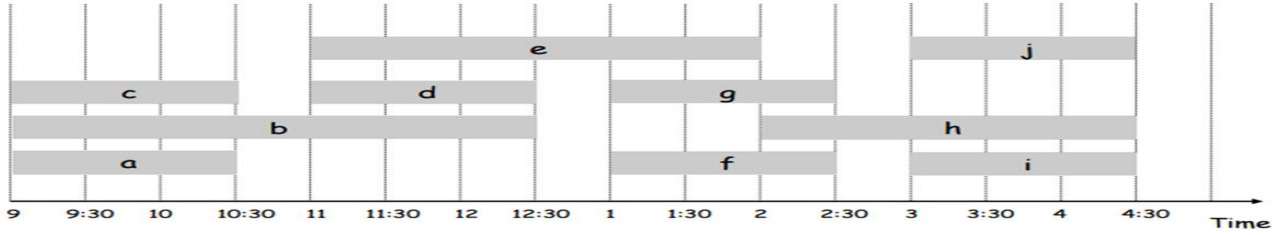
i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	8	9	10	11	12	13	14



An Activity Selection Problem



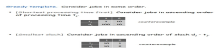




- What is the maximum number of activities that can be completed?
  - {a3, a9, a11} can be completed
  - But so can {a1, a4, a8, a11} which is a larger set
  - But it is not unique, consider {a2, a4, a9, a11}

Ex: This schedule uses only 3.



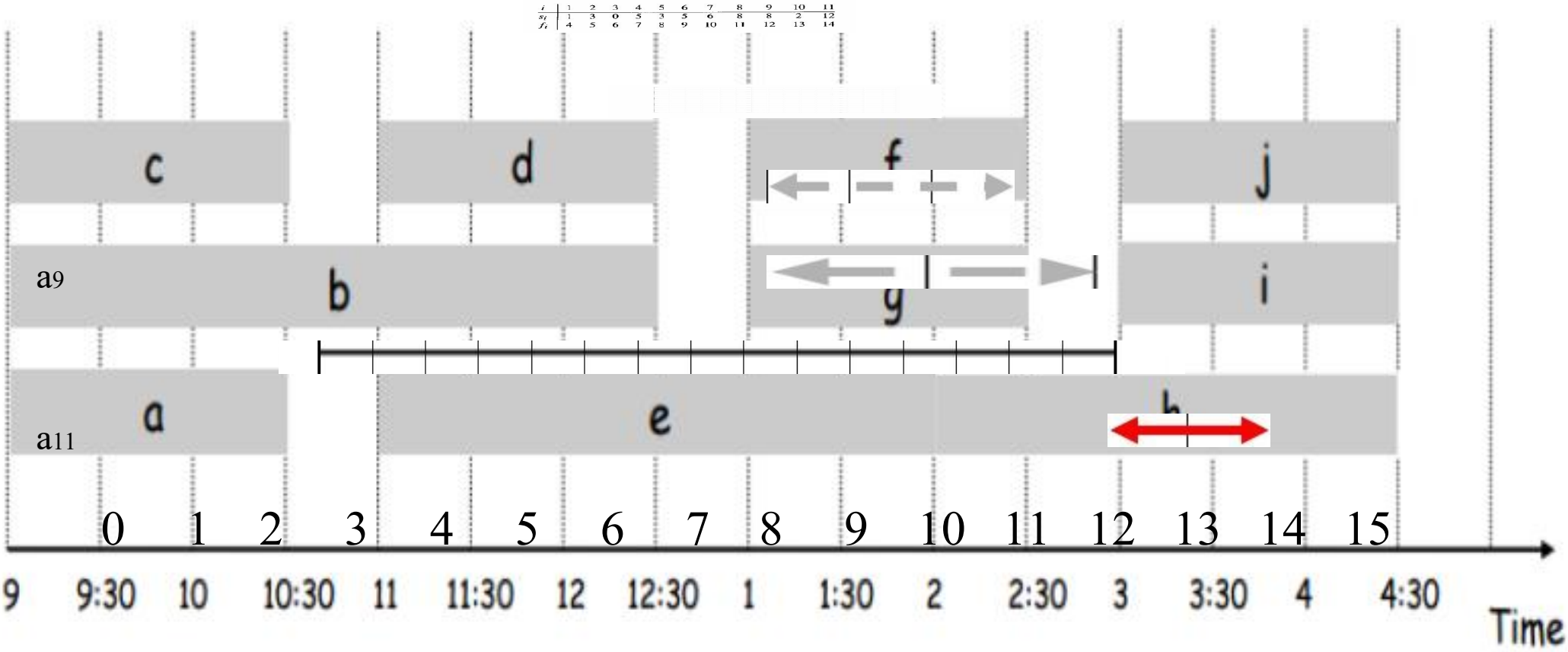


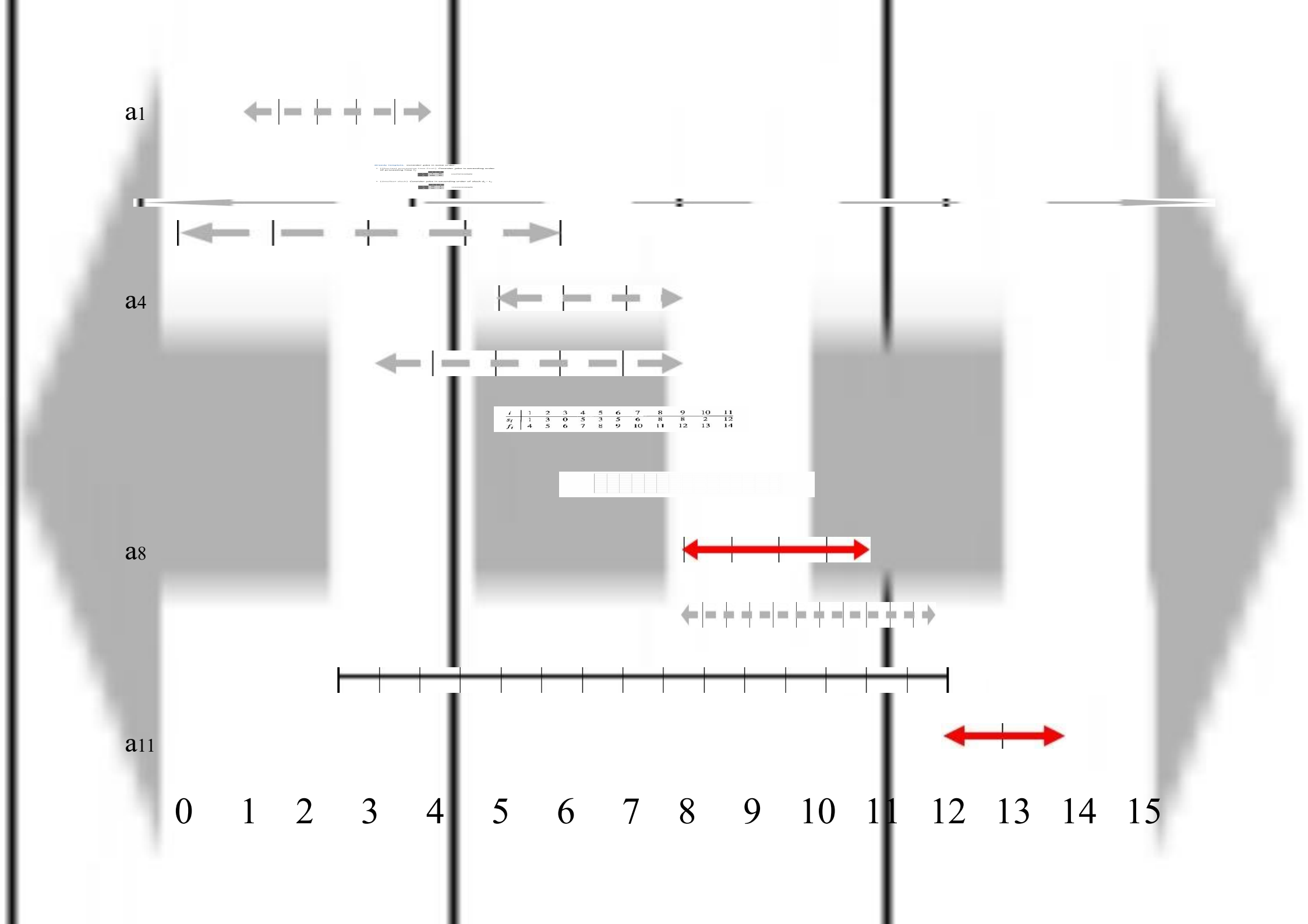
a3

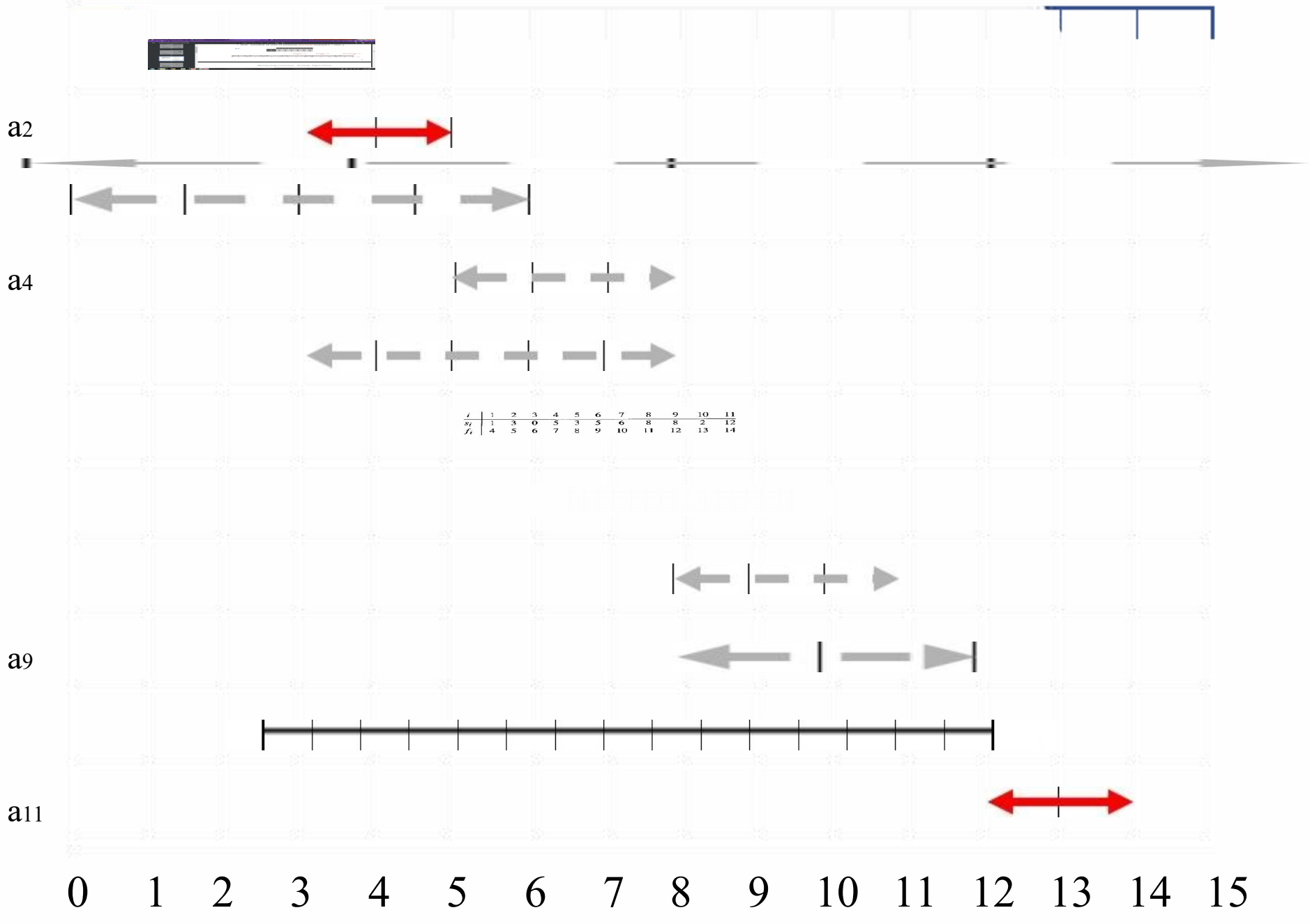




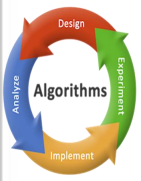





a1
a4
a8
a11
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15


a2
a4
a9
a11
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

An The Optimal Substructure of the A.-S. Problem



- $S_{ij} = \{a_k \in S : f_i \le s_k < f_k \le s_j\}$

  = activities that start after a_i finishes and finish before a_j starts

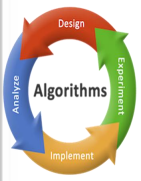


- Activities in S_{ij} are compatible with
  - all activities that finish by f_i, and
  - all activities that start no earlier than s_j
- To represent the entire problem, add fictitious activities:

  $a_0 = [-\infty, 0)$

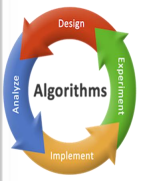
  $a_{n+1} = [\infty, \text{“}\infty+1\text{”})$
  - We don't care about $-\infty$ in a_0 or “$\infty+1$” in a_{n+1}
- Then $S = S_{0, n+1}$
- Range for S_{ij} is $0 \le i, j \le n + 1$

An The Optimal Substructure of the A.-S. Problem



- Assume that activities are sorted by monotonically increasing finish time
  - $f0 \le f1 \le f2 \le \cdots \le fn < fn+1$
- Then $i \ge j \rightarrow Sij = \varnothing$
  - ➔ If there exists a_k S_{ij} : $f_i \le s_k < f_k \le s_j < f_j \rightarrow f_i < f_j$
  - ➔ But $i \ge j \rightarrow f_i \ge f_j$. Contradiction !
- So only need to worry about S_{ij} with $0 \le i < j \le n + 1$
- All other S_{ij} are $\varnothing$

An The Optimal Substructure of the A.-S. Problem

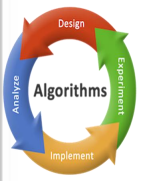


- Suppose that a solution to S_{ij} includes a_k.
  Have 2 subproblems:
  - S_{ik} (start after a_i finishes, finish before a_k starts)
  - S_{kj} (start after a_k finishes, finish before a_j starts)

- Solution to S_{ij}
  - { solution to S_{ik} }⟷{a_k }⟷ { solution to S_{kj} }

- Since a_k is in neither subproblem
  - the subproblems are disjoint
  - | solution to S | = | solution to S_{ik} | + 1 + | solution to S_{kj} |

An The Optimal Substructure of the A.-S. Problem

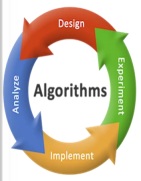


- If an optimal solution to S_{ij} includes a_k , then

the solutions to S_{ik} and S_{kj} used within this solution must be optimal as well

- Let A_{ij} = optimal solution to S_{ij}
- So $A_{ij} = A_{ik} \mapsto \{a_k\} \mapsto A_{kj}$, assuming:
  - S_{ij} is nonempty
  - we know a_k

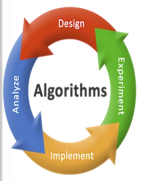
An The Optimal Substructure of the A.-S. Problem



- $c[i, j]$: size of maximum-size subset of mutually compatible activities in S_{ij}
  - $i \geq j \rightarrow S_{ij} = \emptyset \rightarrow c[i, j] = 0$
- If $S_{ij} \neq \emptyset$ suppose we know that a_k is in the subset
  - $c[i, j] = c[i, k] + 1 + c[k, j]$.
- But, we don't know which k to use, and so

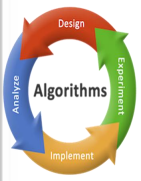
$$c[i, j] = \begin{cases} 0 & \text{if } S_{ij} = \emptyset , \\ \max\limits_{\substack{i<k<j \\ a_k \in S_{ij}}} \{c[i, k] + c[k, j] + 1\} & \text{if } S_{ij} \neq \emptyset . \end{cases}$$

Early Finish Greedy

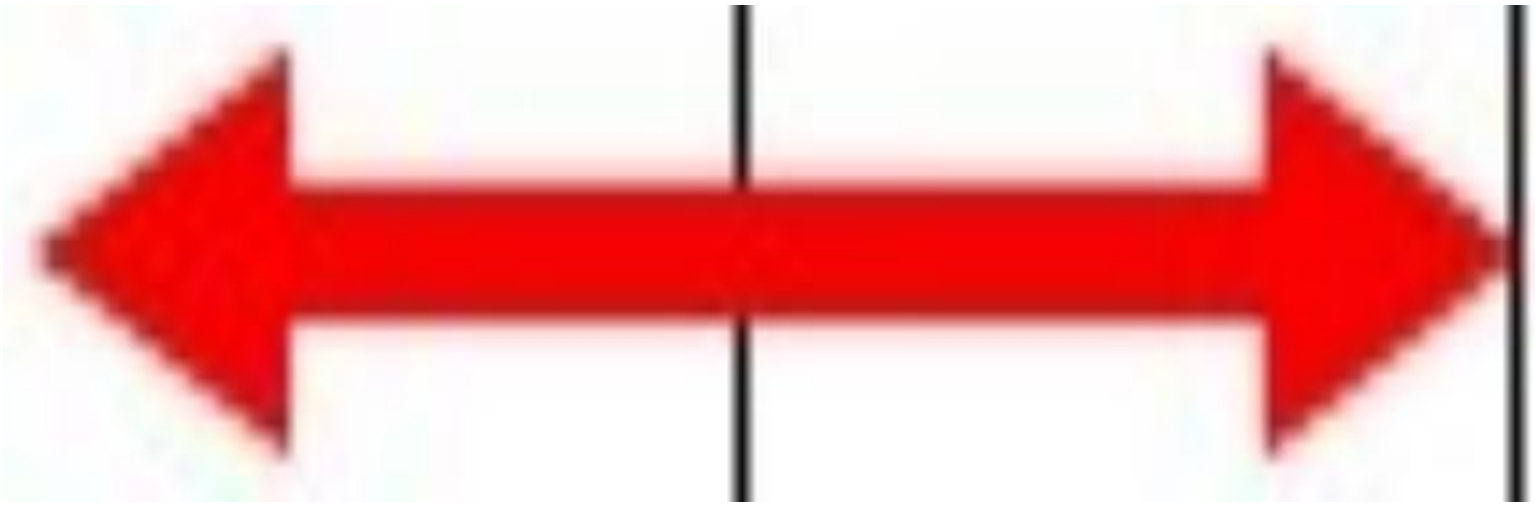


- Select the activity with the earliest finish
- Eliminate the activities that could not be scheduled
- Repeat!

A Recursive Greedy Algorithm

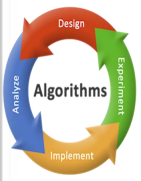


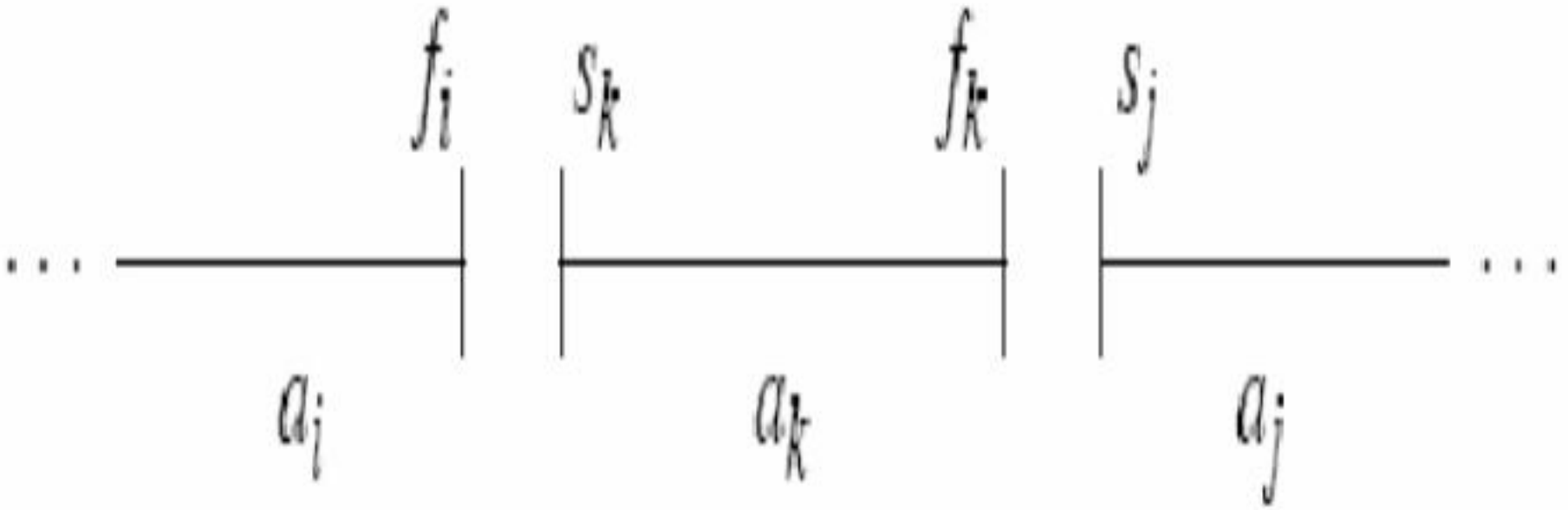
- Assumes activities already sorted by monotonically increasing finish time
  - ➔ If not, then sort in $O(n \lg n)$ time
  - ➔ Return an optimal solution for $S_{i,n+1}$



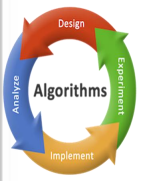
- *Initial call:* REC-ACTIVITY-SELECTOR$(s, f, 0, n)$
- *Time:* $\Theta(n)$ — each activity examined exactly once

Example


Design
Experiment
Analyze
Algorithms
Implement

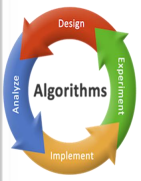

f_i
s_k
f_k
s_j
a_i
a_k
a_j

Example





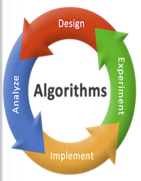
An Iterative Greedy Algorithm





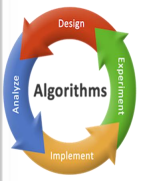
Time $\Theta(n)$

Elements of the greedy strategy



1. Determine the optimal substructure of the problem.
2. Develop a recursive solution. (For the activity-selection problem, we formulated recurrence (16.2), but we bypassed developing a recursive algorithm based on this recurrence.)
3. Show that if we make the greedy choice, then only one sub-problem remains.
4. Prove that it is always safe to make the greedy choice. (Steps 3 and 4 can occur in either order.)
5. Develop a recursive algorithm that implements the greedy strategy.
6. Convert the recursive algorithm to an iterative algorithm.

Greedy versus dynamic programming



- The ***0-1 knapsack problem*** is the following. A thief robbing a store finds n items. The ith item is worth *i* dollars and weighs w_i pounds, where *i* and w_i are integers. The thief wants to take as valuable a load as possible, but he can carry at most W pounds in his knapsack, for some integer W . Which items should he take?
- In the ***fractional knapsack problem***, the setup is the same, but the thief can take fractions of items, rather than having to make a binary (0-1) choice for each item.
  - Calculate value per pound of an item

Greedy Strategy for 0-1?

